

REMAINING SHELF-LIFE ESTIMATION OF FRESH FRUITS AND VEGETABLES DURING TRANSPORTATION

A Project Progress Report

Submitted in partial fulfillment for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

M.TEJASWI (N190910)

P. RAJA KUMARI (N190779)

Under the Esteem Guidance of

Mrs.N.SWATHI



**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
Rajiv Gandhi University of Knowledge Technologies – Nuzvid
Nuzvid, Krishna, Andhra Pradesh – 521202.**



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Krishna, Andhra Pradesh – 521202.

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, “**REMAINING SHELF-LIFE ESTIMATION OF FRESH FRUITS AND VEGETABLES DURING TRANSPORTATION**” is the bonafide work of **M.TEJASWI (ID NO: N190910), P.RAJA KUMARI (ID NO:N190779)**, carried out under my guidance and supervision for 4th year major-project of **Bachelor of Technology** in the department of Computer Science and Engineering under RGUKT IIIT Nuzvid. This work is done during the academic session July 2024 – May 2025, under our guidance.

Mrs.N.Swathi

Assistant professor,
RGUKT, Nuzvid.

Project Examiner

Department of CSE ,
RGUKT,Nuzvid.



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
Rajiv Gandhi University of Knowledge Technologies – Nuzvid
Nuzvid, Krishna, Andhra Pradesh – 521202.

DECLARATION

We certify that

- ✓ The work contained in this report is original and has been done by us under the
- ✓ guidance of my supervisor(s).
- ✓ The work has not been submitted to any other Institute for any degree or diploma.
- ✓ We have followed the guidelines provided by the Institute in preparing the report.
- ✓ We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- ✓ Whenever we have used materials (data, theoretical analysis, figures, and text) from other sources, We have given due credit to them by citing them in the text of the report and giving their details in the references. Further, We have taken permission from the copyright owners of the sources, whenever necessary

Date: 01-05-2025,

Place: Nuzvid.

M.TEJASWI (N190910)

P.RAJA KUMARI (N190779)

ACKNOWLEDGEMENT

We would like to express our profound gratitude and deep regards to our guide **Mrs. N. Swathi** for her exemplary guidance, continuous monitoring, and constant encouragement throughout the course of our B.Tech program. We shall always cherish the time spent with her during this project for the invaluable knowledge gained in the field of **machine learning and real-time predictive systems**.

We are extremely grateful for the confidence she bestowed in us and for entrusting us with the project titled

"Remaining Shelf-Life Estimation of Fresh Fruits and Vegetables During Transportation."

We also extend our sincere thanks to **Mrs. Samineni Bhavani** (Head of the Department, CSE) and all the faculty members for being a constant source of inspiration and support, which played a vital role in the successful completion of this work. Finally, and most importantly, we would like to express our heartfelt thanks to **God** and our **beloved parents** for their blessings, and to our friends for their help, encouragement, and warm wishes.

M.TEJASWI(N190910)

P.RAJA KUMARI (N190779)

INDEX

S.NO.	CHAPTER PAGE	PAGENO
1	ABSTRACT	9
2	INTRODUCTION	10
2.1	WHAT IS DEEP LEARNING?	11
2.1.2	HOW DEEP LEARNING WORKS	12
3	LITERATURE SURVEY	13-14
4	SYSTEM ANALYSIS	15-16
4.1	EXISTING SYSTEM	
4.2	PROPOSED SYSTEM	

5	FUNCTIONAL REQUIREMENTS	17
5.1	SYSTEM FUNCTIONAL REQUIREMENTS	18
6	NON-FUNCTIONAL REQUIREMENTS	19 -20
6.1	REQUIREMENTS	
7	FEASIBILITY STUDY	21
8	REQUIREMENT SPECIFICATION	22
8.1	HARDWARE REQUIREMENTS	
8.2	SOFTWARE REQUIREMENTS	
9	SYSTEM DESIGN	23-26
9.1	METHODOLOGY	
9.2	DATASET	
9.3	SYSTEM ARCHITECTURE	

10	UML DIAGRAMS	27-32
11	TECHNOLOGY DESCRIPTIONS	33 - 35
12	SAMPLE CODE	36 -41
13	TESTING	42-45
13.1	INTRODUCTION	
13.1.1	SYSTEM TESTING AND IMPLEMENTATION	
13.1.2	TESTING TECHNIQUES	
13.1.3	TESTING STRATEGIES	
14	SCREENSHOTS	46-53
15	CONCLUSION	54
16	REFERENCES	57

LIST OF FIGURES

Figure 1 Architecture

Figure 2 Use case Diagram

Figure 3 Class Diagram

Figure 4 Sequence Diagram

Figure 5 Collaboration Diagram

Figure 6 Activity Diagram

LIST OF SCREENS

Screen 1 Home Page

Screen 2 Upload Strawberry Shelflife Dataset

Screen 3 Dataset loaded

Screen 4 Pre-process Dataset

Screen 5 Propose Shelf Life Estimation + ANOVA

Screen 6 Extension CNN SLEM

Screen 7 Model Error Comparison Graph

Screen 8 ANOVA table detail

1.ABSTRACT

Ensuring the accurate prediction of the Remaining Shelf-Life (RSL) for Fresh Fruits and Vegetables (FFVs) during transportation is vital for effective planning and cost management. The Internet of Things (IoT) allows real-time processing of environmental data, but existing RSL models are often qualitative, invasive, or static. This study presents a new, validated RSL model that dynamically estimates general decay rates based on respiration rates, integrating them over time. Unlike previous methods, it is non-invasive and does not require pre-deployment experiments. A simplified surrogate model was also developed to facilitate real-time applications in IoT systems. Testing with various fresh products showed minimal prediction errors, proving the model's reliability

INTRODUCTION

Fresh fruits and vegetables (FFVs) are highly perishable and require careful handling during transportation to maintain quality and reduce waste. Estimating their remaining shelf life (RSL) in real time is crucial for effective supply chain management.

This project aims to develop a **non-invasive, real-time system** to predict the RSL of FFVs using environmental data—mainly temperature—collected during transport. By integrating **statistical methods like ANOVA** and **deep learning models such as CNN**, the system ensures accurate shelf-life predictions, enabling better decision-making and minimizing losses.

This project aims to solve that problem by developing a real-time, non-invasive, and accurate model for predicting the remaining shelf-life of FFVs during transportation. The focus is on creating a system that can use real-time temperature data to calculate how much usable time is left for each product before it starts to spoil. Unlike traditional systems, this approach doesn't require any pre-deployment experiments or invasive testing. The model is designed to adapt to sudden environmental changes, making it ideal for use in transportation vehicles, storage units, or even in smart refrigerators used by retailers

2.1 WHAT IS DEEP LEARNING?

Deep learning is a subfield of AI that engages people's attention the most today. This is the case because deep learning allows the automation of tasks such as image classification, speech recognition, object detection, and content description. In short, deep learning allows machines to comprehend data in a more advanced way.

For example, deep learning is used in image classification, speech recognition, object detection, and content analysis. The exponential growth can be attributed to several factors:

Key Drivers of Deep Learning Advancements

1. Algorithmic Improvements

The innovations in the algorithms have drastically increased the usability of the deep learning models.

2. New Machine Learning Approaches

The methodologies have evolved over the years, and that has retained the prediction accuracy of the models as well as the decisions made.

3. Specialized Neural Networks

New formats of the neural networks have been created to enable the performance of certain functions such as document translation or photos categorization to these networks.

4. Abundance of Metadata

The availability of such data is practically a revolution and includes the following types:

- Streaming data from IoT devices;
- Textual data from social media;
- Professional notes like physicians' notes and investigative transcriptions.

2.2 How Deep Learning Works

The deep learning paradigm has the ability to bring revolution in advancement of the field analytics. Deep learning treats the problem differently, as in this approach instead of being explicitly coded the whole program detailing how the problem can be solved, the clear problem, in this case, is to teach the computer to learn how to solve it by itself.

Analytical Methodology In Deep Learning

In traditional analytics, the steps involved in the approach tend to include the following:

1. **Feature Engineering:** Taking out an variable (features) or picking out variables from the data set and recreating ones that can adequately describe the issue.
2. **Model Development:** All the phases of the study until a particular model is completed and picked on.
3. **Parameter Fitting:** Creating the best model that represents the data by changing the parameters of the model.

Although this approach can yield satisfactory predictive systems, it has weaknesses.

The performance of the system is highly influenced by the quality of the features and the model.

For example, while constructing a model for detecting fraud, one would say:

- You start with parameters that are already set and apply data manipulations in order to generate a model.
- It follows that this process may produce several thousands of variables, which will involve painstaking work to sift through this mass and select the significant ones.
- Inserting new information requires the app

3.LITERATURE SURVEY

1. Cold Chain Risk Management for FFVs (Srivastava et al., 2015)

Srivastava et al. identified risks in the FFV supply chain, such as temperature abuse and logistical inefficiencies. Their research highlights the importance of real-time monitoring to mitigate food losses.

2. Global Food Waste Analysis (Gustavsson et al., 2011)

Gustavsson et al. provided a comprehensive analysis of global food loss, estimating that one-third of edible food is wasted annually. However, their study lacks predictive models for shelf-life optimization.

3. IoT-Based Food Safety System (Bouzembrak et al., 2019)

Bouzembrak et al. introduced an IoT-enabled system for food safety monitoring, utilizing real-time sensors. While effective, the deployment costs of IoT infrastructure remain a challenge.

4. Vehicle Routing in Cold Supply Chain (Awad et al., 2021)

Awad et al. optimized vehicle routing for perishable goods by integrating shelf-life predictions into logistics planning. Their model improves efficiency but is complex to implement in real-world scenarios.

5. Temperature Abuse in Refrigerated Foods (Jol et al., 2006)

Jol et al. investigated the effects of temperature fluctuations on food quality, emphasizing the need for temperature control. However, their approach is limited to temperature monitoring without considering other environmental factors.

6. Statistical Process Control (SPC) for Shelf-Life (Hertog et al., 2014)

Hertog et al. proposed an SPC-based framework for monitoring shelf-life in warehouses. Their approach improves real-time decision-making but requires statistical expertise for effective implementation.

7. Accelerated Storage Data for Shelf-Life Estimation (Corradini & Peleg, 2007)

Corradini and Peleg developed a predictive model using accelerated storage tests to estimate shelf-life. While effective, their approach does not account for dynamic environmental changes.

8. Smart Logistic Unit for Shelf-Life Prediction (La Scalia et al., 2017)

La Scalia et al. introduced a smart packaging solution that integrates sensor data for real-time shelf-life prediction. Despite its accuracy, the high cost of deployment remains a concern.

9. WebGIS-Based Real-Time Shelf-Life Prediction (Sciortino et al., 2016)

Sciortino et al. designed a WebGIS-based system that enables real-time shelf-life estimation. However, their approach requires continuous updates to maintain model accuracy.

10. Respiration and Transpiration in Fresh Foods (Tano et al., 2009)

Tano et al. studied the impact of respiration and transpiration on FFV shelf-life. Their findings emphasize the need for species-specific models due to variability in respiration rates

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

The current systems for predicting the Remaining Shelf-Life (RSL) of Fresh Fruits and Vegetables (FFVs) are limited in effectiveness. While existing models can accurately predict the shelf-life of processed and preserved foods, they struggle with fresh, biologically active products like FFVs. These models often rely on invasive, time-consuming tests, and are highly specific to particular food types, making them impractical for dynamic and varied supply chain conditions. Although IoT technology enables real-time monitoring of environmental factors, the lack of adaptable, non-invasive computational models prevents accurate RSL estimation, leading to significant food waste and profit losses in the supply chain.

Disadvantages of Existing system

1. High profit loss from wastage.
2. Inefficient prediction models for FFVs.
3. Temperature abuse accelerates food decay.
4. Cross-contamination risk during transport.
5. Current models need invasive testing.
6. Time-consuming pre-tests are required.
7. Limited applicability to various foods.

4.2 PROPOSED SYSTEM

To accurately predict shelf life of FFV author of this paper generating FFV shelf life using simulation with and without opening doors and this simulation dataset.

Above dataset contains Time and temperature and by subtracting old temperature with current temperature we can get FFV RSL (remaining shelf life) value. ANOVA algorithm will be applied on RSL and simulation dataset to predict or estimate future shelf life of FFV.

Analysis of variance (ANOVA) is a statistical technique used to check if the means of two or more groups are significantly different from each other. ANOVA checks the impact of one or more factors by comparing the means of different samples.

Advantages of Proposed System

1. Real-time RSL prediction for FFVs.
2. Non-invasive, non-destructive implementation.
3. Reduces food waste and losses.
4. Adapts to dynamic logistic conditions.
5. Supports real-time IoT applications.
6. No need for pre-deployment tests.
7. Enhances planning and cost estimation.
8. Improves supply chain efficiency.
9. Provides accurate shelf-life estimation.
10. Validated with minimal prediction errors..

5.FUNCTIONAL REQUIREMENTS

Functional requirements form an important part in the sphere of software and systems engineering as it determines the conditions which must be fulfilled in order for the system or any of its components to perform their particular tasks. Explain how inputs are transformed into outputs and what the system is expected to achieve once it is completed. Analytic computations, data creation and even some operational processes would be some instances of these requirements. Functional requirements concern the system in its aspects including but not limited to the performance capabilities while non-functional requirements address specific components and capabilities within a system, such as performance, security, or reliability.

Functional requirements are typically put forth as the condition “the system must do ,” whereas for non-functional requirements the statement is put in the form of “the system shall be .” These functional requirements are provided in the system design, while non-functional requirements are maintained in the systems’ architectures. Therefore functional requirements controls the application architecture, and non-functional requirements control the technical architecture.

5. 1 SYSTEM FUNCTIONAL REQUIREMENTS

1. Real-Time Data Acquisition

The system must continuously collect temperature data from IoT sensors installed in the transportation unit or storage environment.

2. Temperature Fluctuation Analysis

The system must calculate and monitor changes in temperature over time to determine the effect on the remaining shelf-life (RSL) of the produce.

3. Shelf-Life Prediction Using ANOVA

The system must apply the Analysis of Variance (ANOVA) model to predict RSL based on historical and real-time temperature data.

4. Deep Learning-Based Prediction (CNN-SLEM)

The system must implement a CNN-based Shelf Life Estimation Model (SLEM) to improve prediction accuracy by optimizing learning from complex temperature patterns.

5. RSL Calculation Algorithm

The system must compute RSL values dynamically by comparing current temperature data with baseline values using a predefined algorithm.

6. Simulation Data Processing

The system must be able to import, preprocess, and analyze simulation datasets (door open/close scenarios) to train and validate models.

7. Error Rate Evaluation

The system must compare predicted RSL with actual values to calculate model accuracy and display error rates (e.g., mean squared error).

8. Graphical Output Display

The system must generate visual graphs showing actual vs. predicted shelf-life to help users easily interpret model performance.

9. User Interface via Tkinter

The system must provide a user-friendly interface using Python's Tkinter for input, processing, and displaying prediction results.

6. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements of a system indicate the maintained quality or response time of a system or application. For instance, a user may be configured to a system and once selected the system is expected to respond within a certain time frame: 3 seconds for 10,000 users concurrent. In a brief statement, these types of requirements explain how the system is going to fulfill its operations.

Just like deadlines in real life, they are a must while working on any product or delivering services. Many would refer to these as constraints, as these do set limitations or boundaries within which the design needs to operate. One notion of philosophy in design states ‘out of sight, out of mind’ and these graphic leaders allow exactly that so the consumer focuses on the primary objectives. Without NFRs, consumers could easily become unsatisfied due to the inefficiency of a product.

6.1 REQUIREMENTS

Performance

The system should provide shelf-life predictions within a few seconds after receiving temperature input, ensuring quick response time suitable for real-time applications.

Scalability

The system should be capable of handling increased data input from multiple sensors or transportation units without a drop in performance.

Accuracy

The deep learning model (CNN-SLEM) must maintain a prediction error rate of less than 1% under standard operating conditions.

Reliability

The system must operate consistently without crashes or data loss during continuous usage over long transport durations.

Usability

The user interface should be intuitive and easy to navigate, even for users with minimal technical background.

Maintainability

The system codebase should be modular and well-documented to support easy maintenance, updates, and debugging.

Portability

The system should run smoothly on standard Windows 10 or 11 operating systems with basic hardware configurations (8 GB RAM, Intel i3 or higher).

Security

If cloud storage or remote access is implemented, the system must ensure secure data transmission using standard encryption protocols.

Robustness

The system must handle unexpected input values or sensor errors gracefully, without crashing or producing invalid outputs.

Extensibility

The system should allow easy integration of additional features in the future, such as support for new sensor types or additional environmental factors like humidity or light.

7. FEASIBILITY STUDY

The feasibility study succinctly addresses whether the envisaged proposal is workable by presenting an outline and cost figures for the project. This phase is very important in determining whether the system is in concurrence with the objectives of the organization and whether it is appropriate. For purposes of carrying out a feasibility study, there need to be always criteria to be satisfied, in this case, that of understanding the system requirements. It covers the following areas:

1. Economic Feasibility:

This assesses the economic worth of the system. The expenditures incurred in the engagement of such R&D activities should be able to cover the cost incurred. The system is constructed in a way that eliminates many unnecessary costs by using free alternatives and restricting tailor made procurements.

2. Technical Feasibility:

This assesses how the system's technical needs correspond to what is available. The completion of the system development avoids many technical specifications to minimize the alteration of the system at the time of implementation.

3. Social Feasibility:

This measures a system in terms of the aptness or readiness of users to use the system. Training and tutorials build users' self familiarity with the system aiding their seamless adoption of the system in their day to day work processes. User encouragement to give feedback increases the chances of enhanced adoption experience.

8.REQUIREMENT SPECIFICATION

8.1 Hardware Requirements

- PROCESSOR - Intel Core i3/i5
- RAM - 8 GB
- HARD DISK (SSD) - 512 GB

8.2 Software Requirements

- Operating system : Windows 11/ 10
- Coding Language : PYTHON, Jupyter Notebook

9 SYSTEM DESIGN

9.1 METHODOLOGY

1. Data Collection and Description

The **simulation dataset** for estimating **FFV shelf life** is sourced from **Mendeley Data** (<https://data.mendeley.com/datasets/jb8bdstwwb/1>). The dataset records **time and temperature changes** under different conditions, including **door opening and closing** scenarios, which impact FFV longevity.

Key features in the dataset:

- **Time** – Timestamp of each measurement.
- **Temperature** – Current temperature recorded inside the transport unit.
- **RSL (Remaining Shelf Life)** – Estimated shelf life computed by subtracting past temperature values from current ones.

2. Data Preprocessing

Before training the models, the dataset undergoes preprocessing to improve accuracy and reliability:

- **Missing value handling** – Removes incomplete data points.
- **Feature extraction** – Computes RSL by analyzing **temperature changes** over time.
- **Normalization** – Standardizes temperature variations for uniformity.
- **Dataset splitting** – The dataset is divided into **training (80%) and testing (20%) sets**.

3. Shelf Life Estimation Using ANOVA

The **Analysis of Variance (ANOVA)** method is employed to estimate FFV shelf life.

ANOVA is a statistical model that analyzes variations within datasets and determines significant differences.

Steps:

1. The cleaned dataset is fed into the **ANOVA model**.
2. The model examines the impact of **temperature fluctuations on RSL**.
3. Predictions for future **shelf life values** are made based on **historical temperature trends**.
4. The model **error rate** is calculated by comparing predicted shelf life with observed values.

Performance Evaluation:

- **Model accuracy** is assessed by computing the **error rate** (difference between actual and predicted shelf life).
- **Visualization:** The model generates graphs comparing original **vs.** predicted shelf life, where x-axis represents test samples and y-axis represents shelf life.

ANOVA Model Results:

- Achieved **30% error rate**.

4. Deep Learning Extension – CNN-Based SLEM Model

To reduce prediction errors, an extension using a **Convolutional Neural Network (CNN)-based Shelf Life Estimation Model (SLEM)** is proposed.

CNN SLEM Implementation Steps:

1. The **CNN model** is trained using **the same dataset** as ANOVA.
2. Multiple layers of **neurons optimize feature extraction** from temperature variations.
3. The trained model **predicts RSL with improved accuracy**.
4. Performance is evaluated by computing the **new error rate**.

CNN SLEM Model Results:

- Achieved a **0.26% error rate**, significantly lower than the **30% error** of the ANOVA model.

Results

- **ANOVA model achieved a 30% error rate**, indicating moderate accuracy in predicting FFV shelf life.
- **CNN-based SLEM model significantly outperformed ANOVA**, reducing the error rate to **0.26%**.

9.2 DATASET:

To accurately predict shelf life of FFV author of this paper generating FFV shelf life using simulation with and without opening doors and this simulation dataset can be downloaded from below URL

<https://data.mendeley.com/datasets/jb8bdstwwb/1>

Above dataset contains Time and temperature and by subtracting old temperature with current temperature we can get FFV RSL (remaining shelf life) value. ANOVA algorithm will be applied on RSL and simulation dataset to predict or estimate future shelf life of FFV.

9.3 SYSTEM ARCHITECTURE:

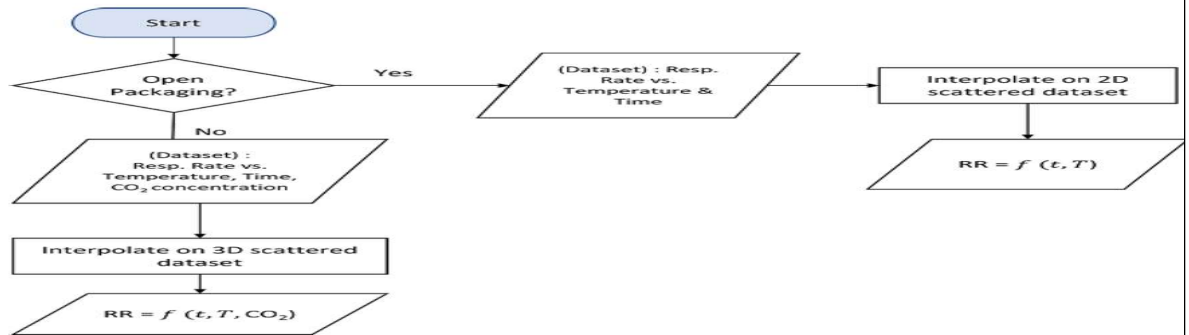


Figure 1 Architecture

10 .UML DIAGRAMS

UML is a model while UML is a modeling language. Unified Modeling Language is an internationally accepted modeling palette which makes it easy to visually express plans, implemented ideas and facts in terms of a system- OO. It is a language, so it can be spoken and written. It is a standardized language and there is no national ownership.

UML comprises of two basic aspects: a Meta-model and Notation. This has further enabled the specification, visualization, construction and documentation of software artifacts. It moreover fosters business modeling and other non-software systems, placing emphasis graphical notations for better communication of designs of software systems.

UML adopts the best engineering practices which have been found useful in the modeling of complex and large systems. It is a must in object oriented software engineering for the entire process to be clear, consistent and understandable.

Goals of UML

The primary objectives of UML include:

1. Enhancing the ability to create and communicate meaningful models by providing an appropriate visual modeling language.
2. Introducing provisions for the extendibility and specialization of the basic concepts.
3. Guaranteeing freedom with respect to particular programming languages and specific development methodologies.
4. Providing a helpful underpinning to make clear the language.
5. Facilitating the development of the market of the object-oriented tools.
6. Making it possible to use more sophisticated development concepts such as collaborations, frameworks, patterns and components.

10.1 USE CASE DIAGRAM

A use case diagram in Unified Modeling Language (UML) belongs into class of behavioral diagrams and is obtained through use-case analysis. It graphically represents the functionality of the system by depicting the interactions of the actors towards the achievement of their goals which are modeled as use cases. And, it provides an overview of the dependencies between these use cases.

The main objective of use case diagrams is to indicate what tasks the system performs for each of its actors, so that, the tasks and responsibilities of the actors in the system are better understood.

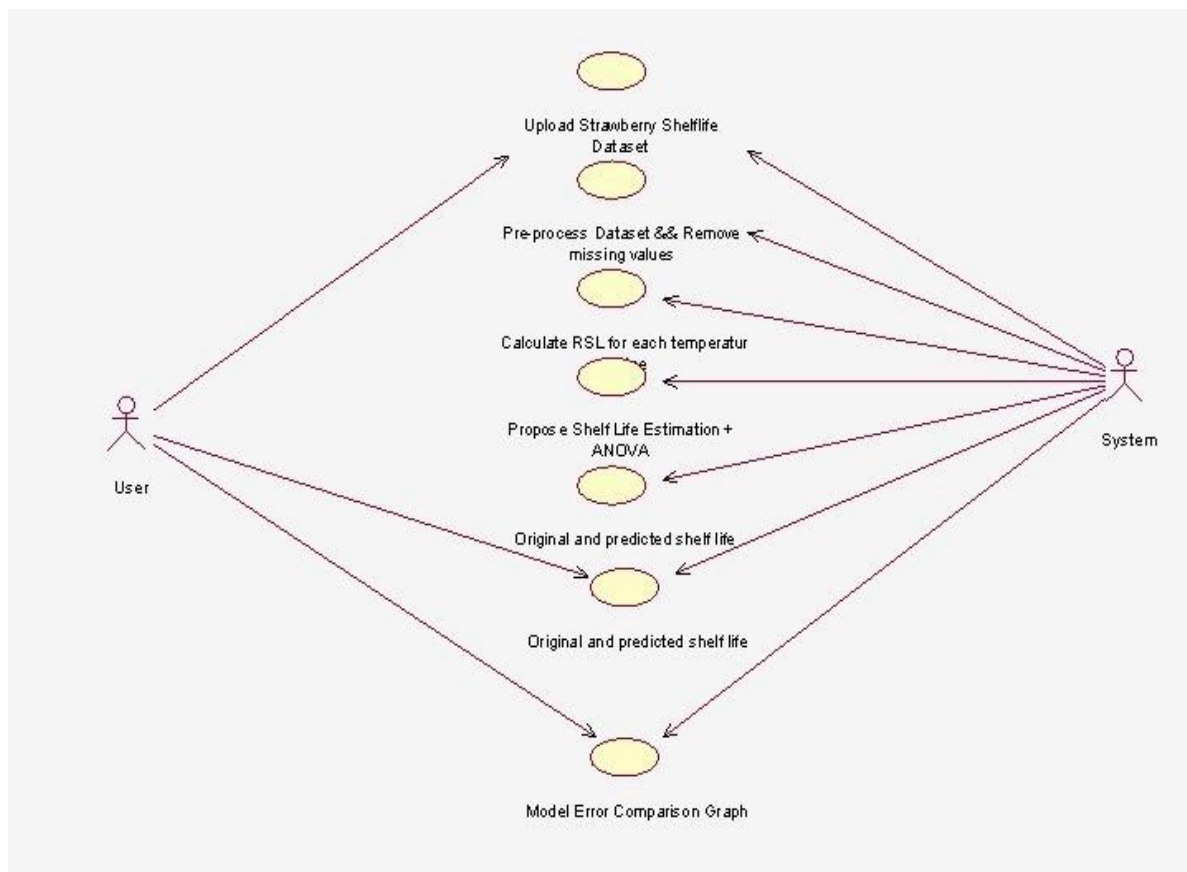


Figure 2 - Use Case Diagram

10.2 CLASS DIAGRAM

As it belongs to specific types within the Unified Modeling Language (UML), a class diagram is categorized as a static structure diagram in computer software construction. This is a system modeling method that models a particular system by displaying its classes (or the sets of objects), their features, methods (or functions), and their inter-relationships.

Class diagrams portray the containment and interaction of classes and provide no ambiguity about the system's blueprint. This is why they are a valuable resource for the analysis and design of object-oriented software systems.

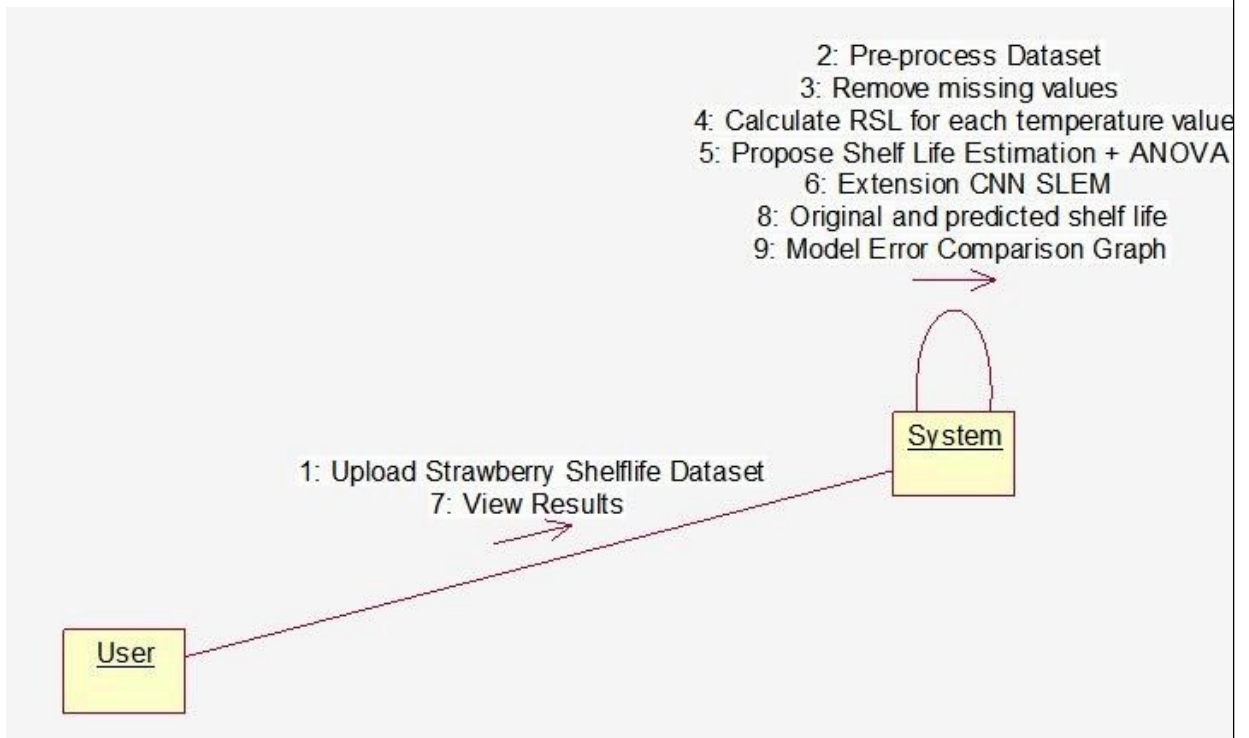


Fig 5- Collaboration Diagram

10.5 ACTIVITY DIAGRAM:

The activity diagram also contains cases of the communication and interactions of objects for the functional purpose. They include a variety of cases such as events and actions in terms of how they flow as well as how they reach the various systems edges or items.

UML activity diagrams outline the business requirements of a process from the user perspective in high level details and also the operational processes of the system explaining how different functional units operate in the context of sequential stages. They are a logically inseparable portion of each process model considering the order of operations and making decisions at each logical point associated with movement of information.

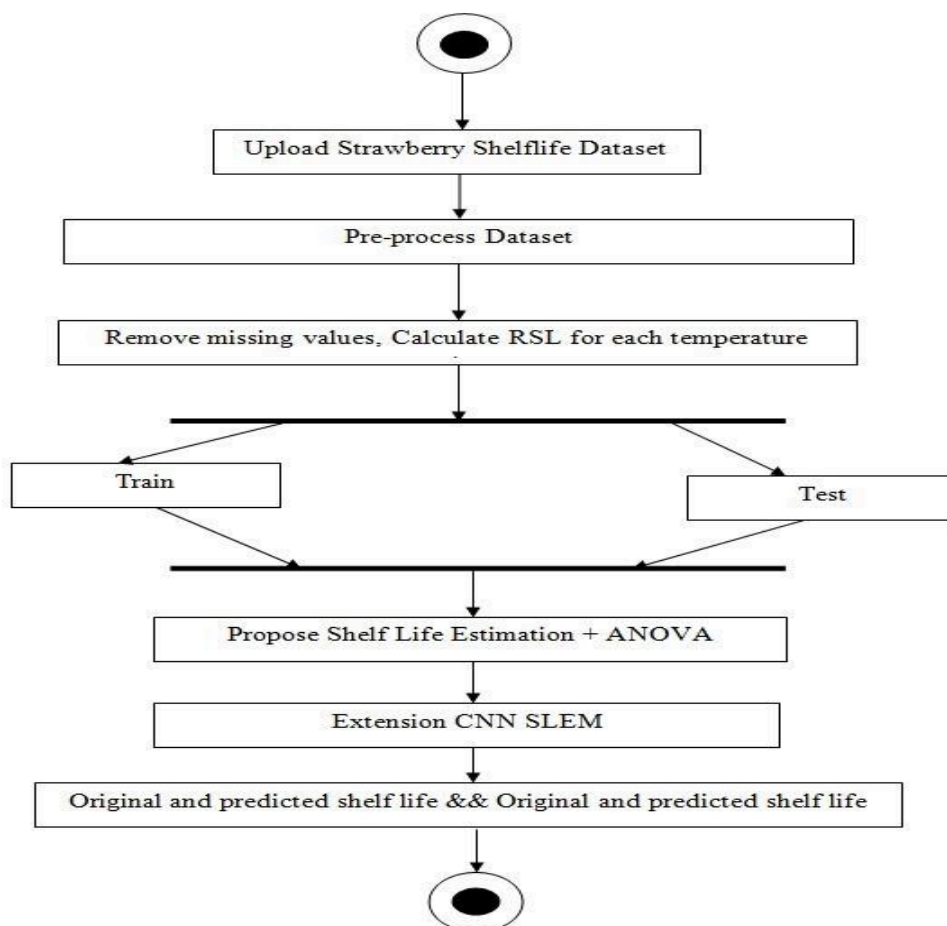


Fig -7 Activity Diagram

11 .TECHNOLOGY DESCRIPTIONS

Python Introduction

Python is general purpose high level dynamic interpreted programming language that is used for many applications. Python programming requires a different set of skills since it supports the object oriented programming approach. Python is clear, straightforward, and offers a number of high level data structures making it a preferred language for programmers. The syntax of the Python language is easy to understand. Its dynamic typing combined with the interpreted nature allows quick jump in application development and scripting. It provides support for multiple paradigms including object oriented, imperative, functional, as well as procedural programming styles to accommodate different project requirements.

11.1 PYTHON FEATURES

Python Characteristics Easy to learn and easy to use:

Being beginner oriented and developer 100 percent focused, python is relatively easy to learn and use.

Expressive Language:

The coding language of Python is declared as highly expressive as it is claimed that it is easy to read and to write.

Interpreted Language:

The reason for this is because Python makes use of an interpreter which takes in the line of code and executes it although it may bring out a few challenges in the debugging of codes.

Cross-platform Compatibility:

This goes to show that the system has the capacity of being used in different kinds of operating systems such as Windows, Linux, Unix and mac operating systems.

Free and Open Source:

Any person who wishes to avail Python can find it free for downloading and It is easy to acquire and modify the source code.

Object-Oriented Programming:

Object-oriented programming is achievable using Python as it provides support for classes and objects among other implementations.

Extensible:

It becomes possible to extend various functionalities and integrate performance improvements by using C/C+= languages with Python environments.

Large Standard Library:

If many different types of modules and functions are needed to develop applications quickly, then Python standard library can be a good solution.

GUI Programming Support:

It is possible to use Python to program graphical user interfaces.

Integrated with Other Languages:

The compatibility and flexibility of the programming language Python is higher as it can work with other languages particularly C, C++, Java languages.

11.2 PYTHON APPLICATIONS

Since Python is a multipurpose language, it can be used in almost all spheres of software development. Other notable fields where Python has its application include:

1. web Applications:

With its libraries for internet protocols as HTML, XML, JSON, and many others, and frameworks such as Django, Flask, Pyramid, Python is among the most popular languages for web development. Such include PythonWikiEngines, Pycoco and others.

2. Desktop GUI Applications:

Desktops, GUI can be developed using Tkinter, wxWidgets, Kivy and PyQt. For multi touch applications across platforms Kivy is good for such purpose.

3. Software Development:

Programming in Python is widely used in software development as an auxiliary language for such activities as build control, testing, management, etc.

4. Scientific and Numeric Computing:

Libraries like SciPy, Pandas, and IPython are popular for scientific computing, engineering, and mathematical analysis.

5. Business Applications:

Python is used for creating ERP and e-commerce solutions such as Tryton.

6. Console-Based Applications:

Tools like IPython are developed using Python for interactive computing.

7. Audio/Video Applications:

Python can develop multimedia apps like TimPlayer or cplay.

8. 3D CAD Applications:

Tools like Fandango are used to create CAD applications.

13.SAMPLE CODE

```
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
from tkinter.filedialog import askopenfilename
from tkinter import ttk
#importing require python packages
import os
import numpy as np
import pandas as pd
from sklearn import linear_model
from scipy import stats
import matplotlib.pyplot as plt #use to visualize dataset vallues
import statsmodels.api as sm
from statsmodels.formula.api import ols
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score
from keras.models import Sequential
from keras.layers import Dense, Activation
from keras.layers import Dropout, LSTM
from keras.callbacks import ModelCheckpoint
import os
import pickle
from keras.layers import MaxPooling2D
from keras.layers import Flatten
from keras.layers import Convolution2D
from sklearn.metrics import mean_squared_error
from math import sqrt
```

```

main = Tk()
main.title("Remaining Shelf-Life Estimation of Fresh Fruits and Vegetables During
Transportation")
main.geometry("1300x1200")
global X, Y, dataset, propose_error, extension_error, scaler, scaler1
def uploadDataset():
    global dataset
    filename = filedialog.askopenfilename(initialdir="Dataset")
    text.delete('1.0', END)
    text.insert(END,filename+" loaded\n\n");
    dataset = pd.read_csv(filename)
    text.insert(END,str(dataset.head()))
def preprocessDataset():
    global dataset, X, Y
    text.delete('1.0', END)
    dataset['Diff'] = dataset['Temp'].diff()
    dataset.fillna(0, inplace = True)
    data = dataset.values
    X = data[:,0:2]
    Y = data[:,data.shape[1]-1]
    text.insert(END,"Dataset Processing RSL (Remaining Shelf Life) Calculation
Completed\n\n")
    text.insert(END,"RSL = "+str(dataset['Diff']))
def SLEMANOVA():
    global X, Y, propose_error, dataset
    text.delete('1.0', END)
    X = sm.add_constant(X) # adding a constant
    olsmod = sm.OLS(Y, X).fit()
    predict = olsmod.predict(X)
    propose_error = sqrt(mean_squared_error(Y, predict.ravel()))
    lm = ols(formula = 'Diff ~ C(Temp)',data=dataset).fit()
    table = sm.stats.anova_lm(lm, typ=2)
    print("ANOVA Summary\n"+str(olsmod.summary())+"\n\n")

```

```

print(str(table)+"\n\n")
text.insert(END,"Propose SLEM Model Error : "+str(propose_error)+"\n\n")
for i in range(0, 10):
    original = Y[i]*30
    predicted = predict[i]*30
    if original > 50:
        original = original / 10
    if predicted > 50:
        predicted = predicted / 10
    text.insert(END,"Observed Life : "+str(int(abs(original)))+ " Days Predicted Life : "+str(int(abs(predicted)))+ " Days\n")
plt.figure(figsize=(5,3))
plt.plot(Y, color = 'red', label = 'Observed Life')
plt.plot(predict, color = 'green', label = 'Predicted Life')
plt.title('Propose SLEM ANNOVA Prediction Graph')
plt.xlabel('Test Data')
plt.ylabel('Life Values')
plt.legend()
plt.show()
#function to calculate accuracy and prediction sales graph
def calculateMetrics(algorithm, predict, test_labels):
    global extension_error
    extension_error = sqrt(mean_squared_error(test_labels, predict))
    predict = predict.reshape(-1, 1)
    predict = scaler1.inverse_transform(predict)
    test_label = scaler1.inverse_transform(test_labels)
    predict = predict.ravel()
    test_label = test_label.ravel()
    text.insert(END,"Extension CNN Model Error : "+str(extension_error)+"\n\n")
    for i in range(0, 10):
        original = test_label[i]*30
        predicted = predict[i]*30
        if original > 50:

```

```

original = original / 10
if predicted > 50:
predicted = predicted / 10
text.insert(END,"Observed Life : "+str(int(abs(original))+" Days Predicted Life :
"+str(int(abs(predicted))+" Days\n")
plt.figure(figsize=(5,3))
plt.plot(test_label, color = 'red', label = 'Observed Life')
plt.plot(predict, color = 'green', label = 'Predicted Life')
plt.title(algorithm+' SLEM Prediction Graph')
plt.xlabel('Test Data')
plt.ylabel('Life Values')
plt.legend()
plt.show()
def CNNSLEM():
global X, Y, extension_error, scaler, scaler1
text.delete('1.0', END)
scaler = MinMaxScaler(feature_range = (0, 1))
scaler1 = MinMaxScaler(feature_range = (0, 1))
X = scaler.fit_transform(X)
Y = Y.reshape(-1, 1)
Y = scaler1.fit_transform(Y)
#now train & plot CNN crop yield prediction
X = np.reshape(X, (X.shape[0], X.shape[1], 1, 1))
#training CNN model
cnn_model = Sequential()
cnn_model.add(Convolution2D(32, (1, 1), input_shape = (X.shape[1], X.shape[2],
X.shape[3]), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
cnn_model.add(Convolution2D(32, (1, 1), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
cnn_model.add(Flatten())
cnn_model.add(Dense(units = 256, activation = 'relu'))
cnn_model.add(Dense(units = 1))

```

```
cnn_model.compile(optimizer = 'adam', loss = 'mean_squared_error')
if os.path.exists("model/cnn_weights.hdf5") == False:

model_check_point = ModelCheckpoint(filepath='model/cnn_weights.hdf5', verbose =
1, save_best_only = True)
cnn_model.fit(X, Y, batch_size = 8, epochs = 1000, validation_data=(X, Y),
callbacks=[model_check_point], verbose=1)
else:
cnn_model.load_weights("model/cnn_weights.hdf5")
predict = cnn_model.predict(X)
calculateMetrics("Extension CNN", predict, Y)#call function to plot LSTM crop yield
prediction
```

13. TESTING

13.1 INTRODUCTION

Software engineers draw a distinction between software fault and software failure. A fault will mean an error that is either a bug in programming or a defect in a certain program's semantic. A fault may or may not result in a failure. A failure on the other hand is such that a software has not performed as expected by the end user. Under certain conditions such as when the faulty code is run on the CPU, or even when it is migrated to another hardware machine or when it is enhanced with new capabilities, faults become failures.

Testing a software application is always essential in the process of analyzing the software with the aim of validating its quality and providing the stakeholders with relevant information. It states the errors that it contains and so guarantees that a software has a certain amount of standards that it fulfills with users and system requirements.

13.1.1 System Testing and Implementation

System testing is concerned with testing the logic of individual module codes and debugging them before their assembly into larger subsystems. This involves:

1. **Module Integration Testing:** This tests the linkages of modules to one another.
2. **System Testing:** This is done to check that the whole system conforms to the requirements provided.
3. **Acceptance Testing:** This goal is to represent the abilities of the system to the customers.

There are two methods that could be used in selecting test cases which include:

- **Black Box Testing** - Testing of the external behavior with respect to the system requirements specification.
- **White Box Testing** - Concentrates on the assessment of internal logic and aims at coverage of the code.

13.1.2 Testing Techniques

Testing discovers mistakes made in the software by executing the software under conditions that were planned in advance. Two main approaches are:

1. Black Box Testing:

Emphasizes functional specification and assesses external behavior without any interference on internal logic. It detects errors such as interface error, the database access error, performance as well.

2. White Box Testing:

Deals with the internal logic by scrutinizing module flow graphs and analyzing logical paths, loops and data structures to be tested

13.1.3 Testing Strategies

To ensure their applications are free from errors and comply with the required functionality, the effective performance of software testing is paramount. Given below are the top primary kinds of software testing strategies used in the software development process:

Unit Testing

Unit testing tends to assure the correctness of the implementation of individual units or components of the software. It is usually conducted to ensure the correctness of internal logic, and that for a particular input element expected output is given. The unit testing is said to be structural and invasive since it depends on the knowledge of the structure of the software developed.

The main objective of developers when using unit testing is to ensure the validity of each decision branch and each internal code path. This guarantees all business logic and configurations function as intended. Unit testing is usually done at the level of components and it helps in detecting faults very early in the development cycle.

Integration Testing

Integration testing is usually done after unit testing and it is concerned with putting several units or components together into one cohesive system as they were meant to work. It is event driven, concentrating on functional interconnections and ultimately the interaction of the components of the system.

While unit testing touches on the performance of each individual component, unit testing is aimed at confirming that all the component systems do not create problems when put together. These tests determine whether there are discrepancies in attached data and also the communication lines between modules so as to enhance unity and harmony in system execution.

System Testing

As the name states, system testing tests the final, integrated software product to ensure everything is functioning according to the requirements. It deals with the global layout and just like the end to end testing, guarantees normal results given the particular components are used in a realistic way.

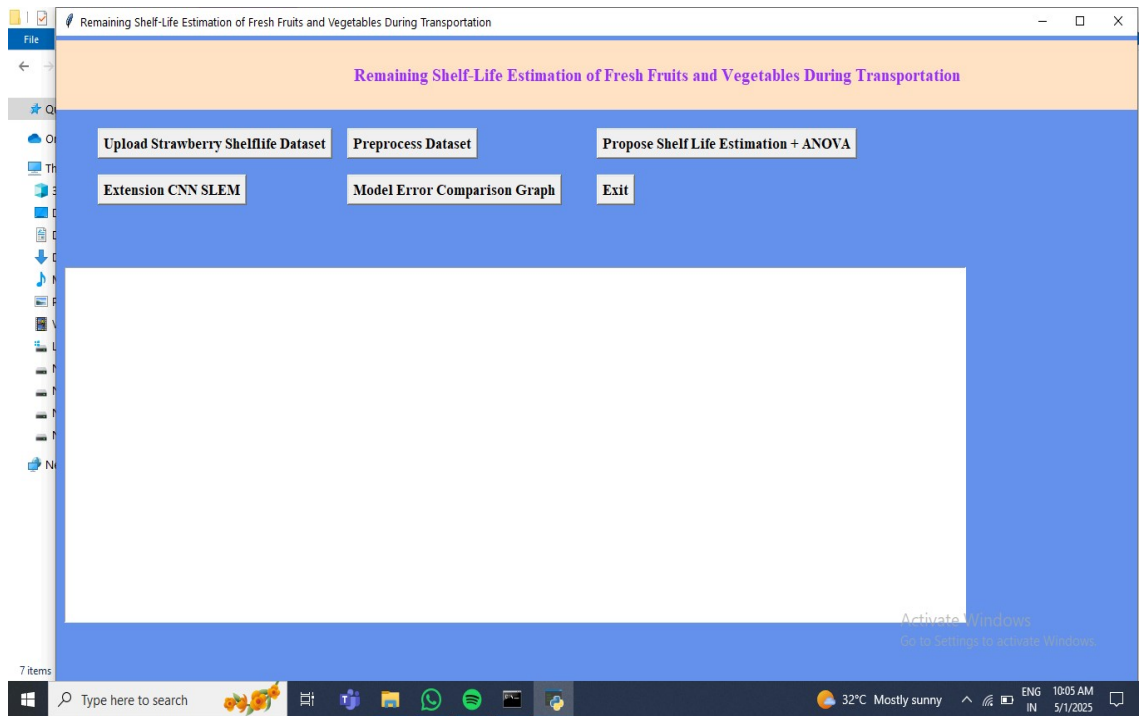
One example of system testing is configuration-oriented system integration testing, which checks how processes interact at various integration points. System testing focuses on process flows and assesses whether the system is reliable and meets the required expectations.

Functional Testing

Functional testing establishes whether the system implemented meets the business and technical requirements stated in system documents and users' manuals. It offers a systematic approach for verifying that all required functions have been implemented correctly and as expected by users.

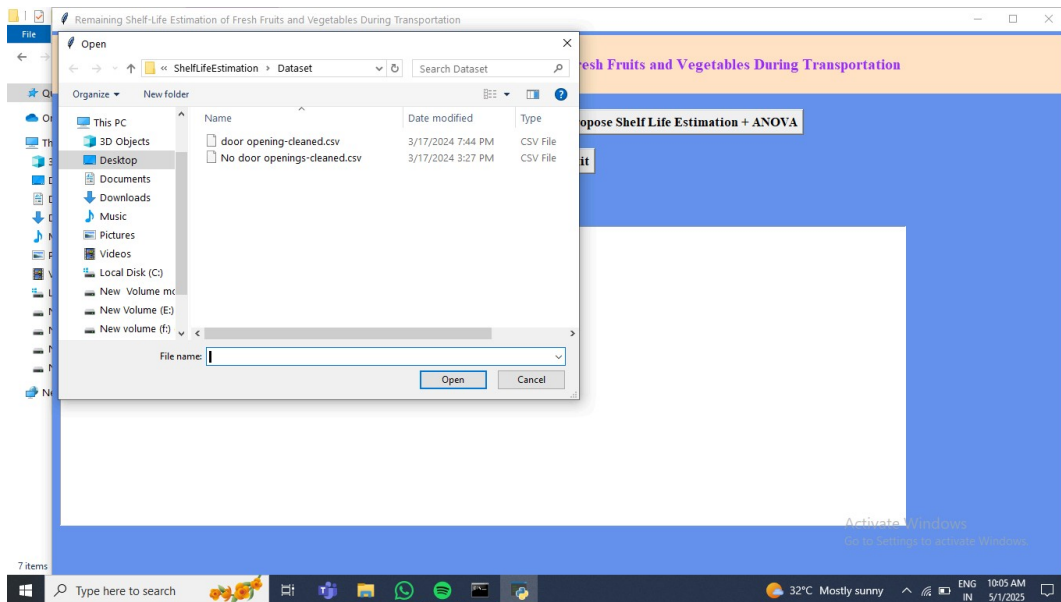
Functional testing was done by using each business process or feature and user interaction against documented requirements. This helps ensure that all features work as they are intended to work without any problems.

15 .SCREENSHOTS

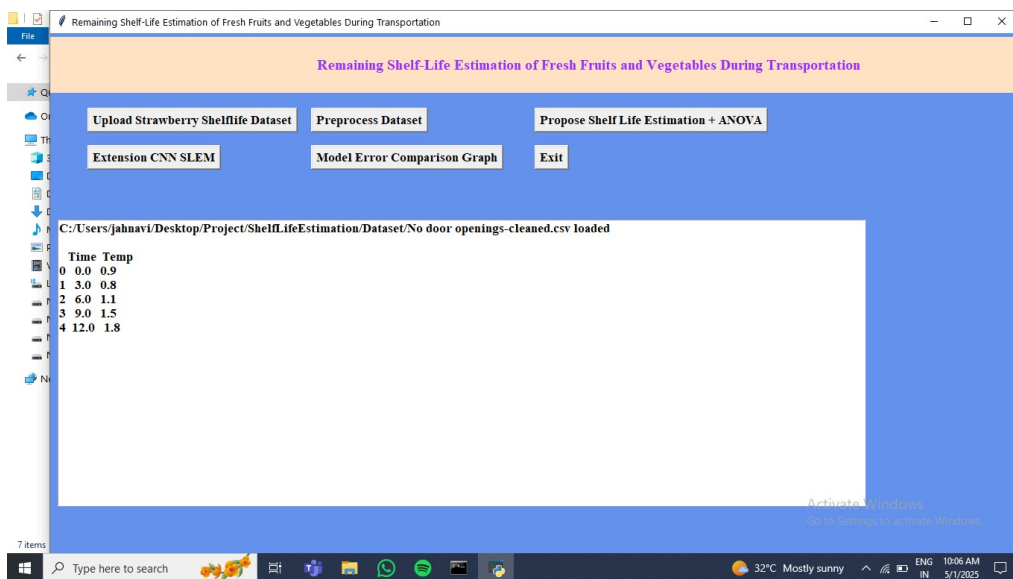


Screen 1 Home Page

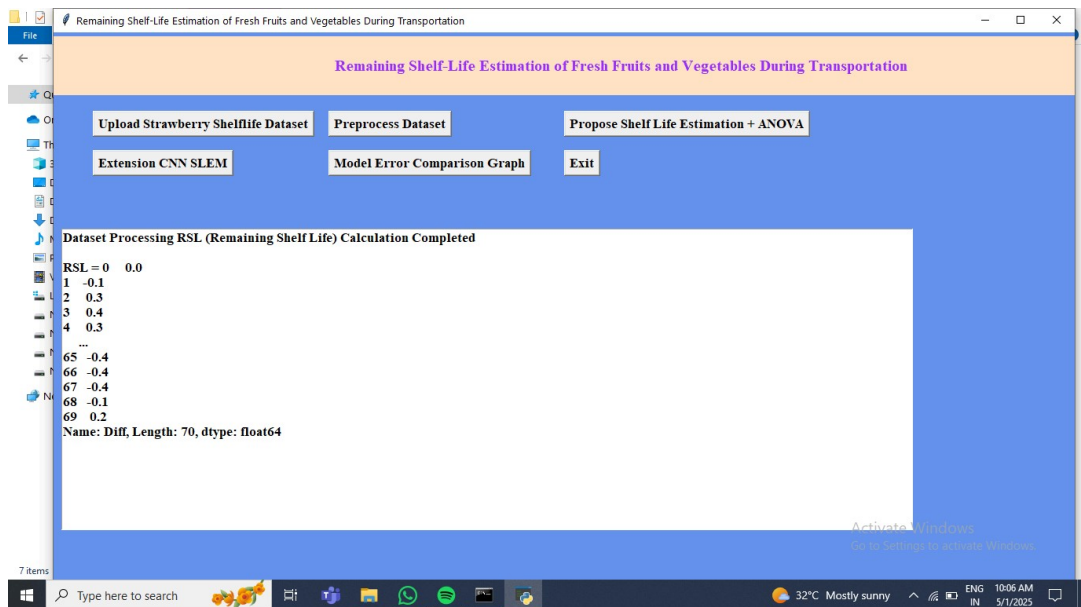
Screen 2 Upload Strawberry Shelflife Dataset



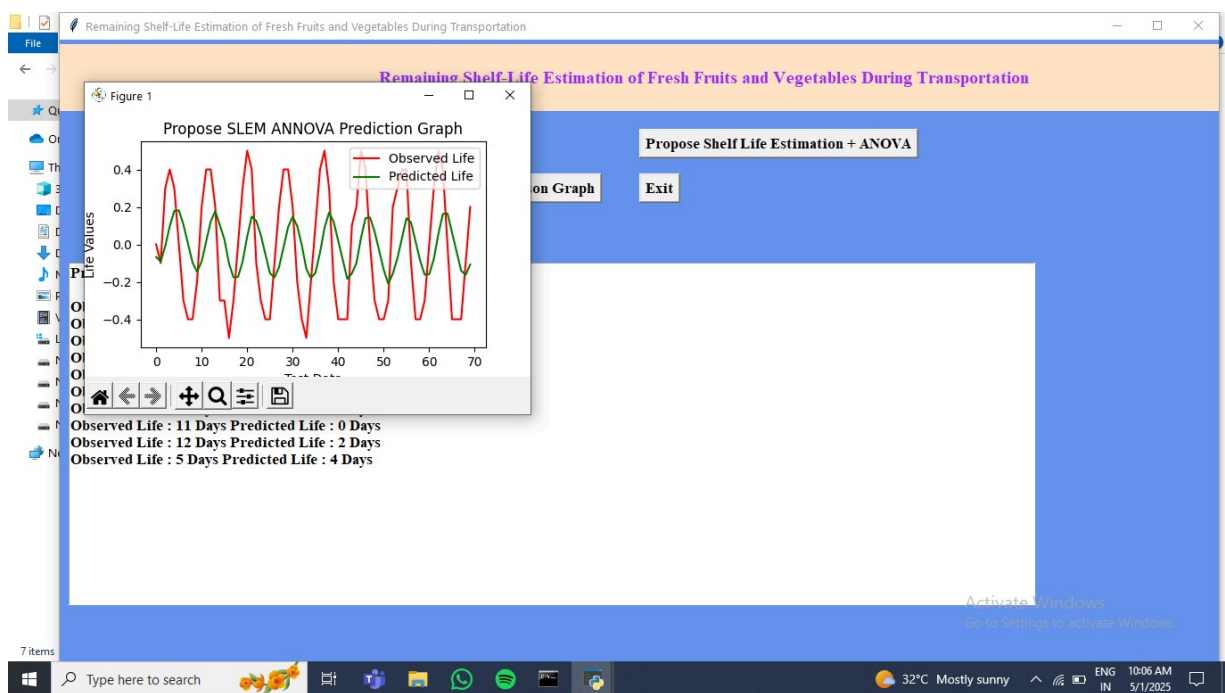
Screen 3 Dataset loaded



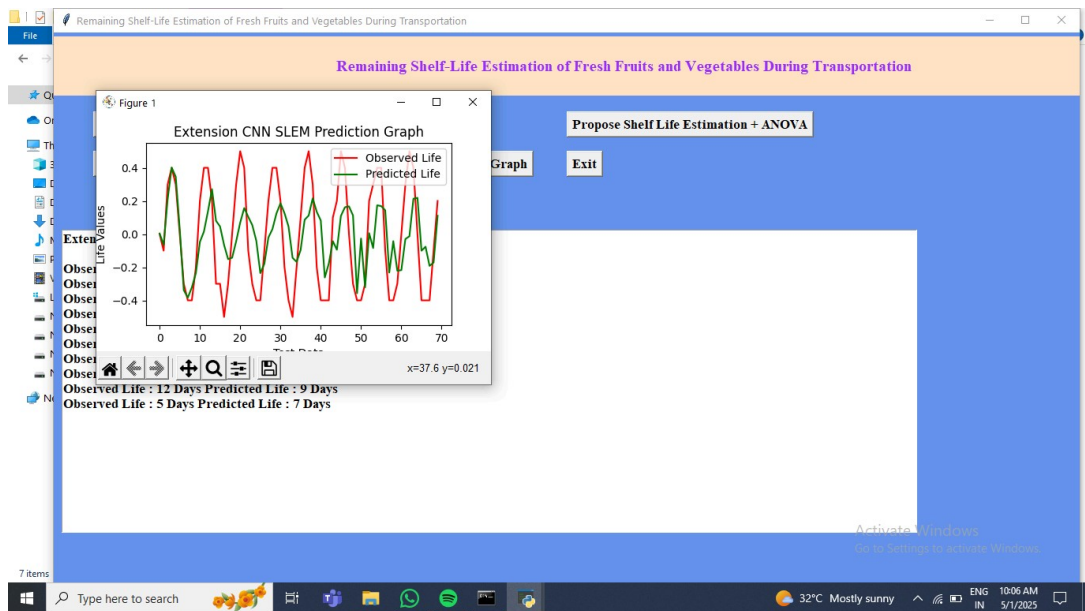
Screen 4 Pre-process Dataset



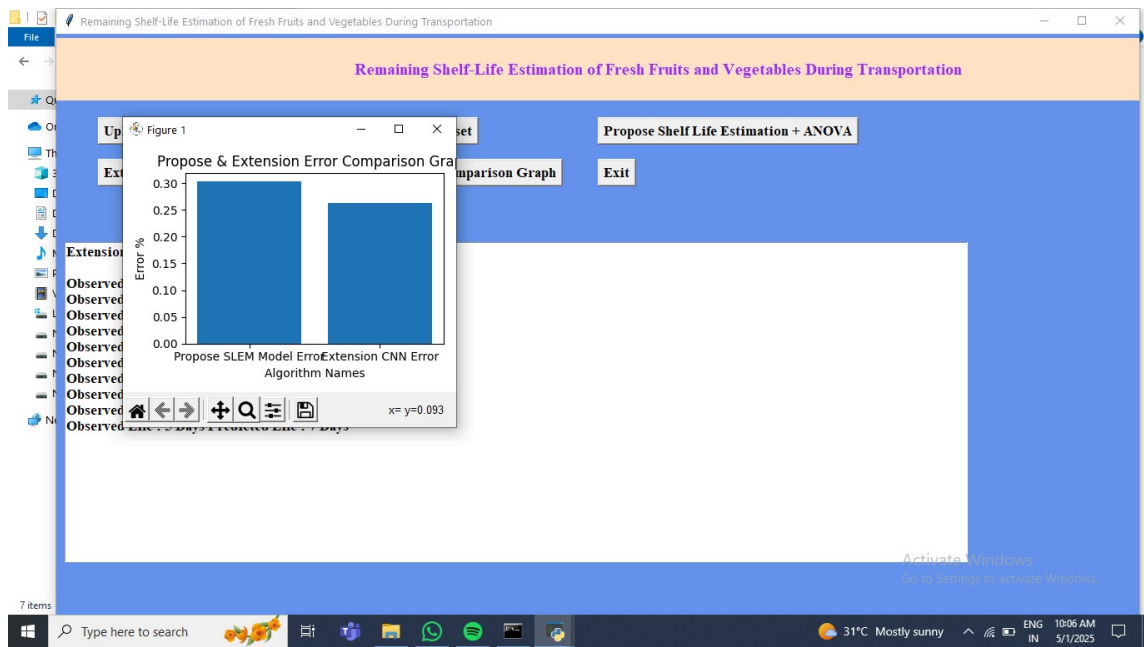
Screen 5 Propose Shelf Life Estimation + ANOVA



Screen 6 Extension CNN SLEM



Screen 7 Model Error Comparison Graph



Screen 8 ANOVA table details

```
C:\Windows\system32\cmd.exe
np_resource = np.dtype([('resource', np.ubyte, 1)])
ANOVA Summary

=====
OLS Regression Results
=====
Dep. Variable: y R-squared: 0.132
Model: OLS Adj. R-squared: 0.106
Method: Least Squares F-statistic: 5.003
Date: Thu, 01 May 2025 Prob (F-statistic): 0.00873
Time: 09:58:20 Log-Likelihood: -15.814
No. Observations: 70 AIC: 37.63
Df Residuals: 67 BIC: 44.37
Df Model: 2
Covariance Type: nonrobust
=====
coef std err t P>|t| [0.025 0.975]
-----
const -0.3027 0.123 -2.452 0.017 -0.549 -0.056
x1 0.0009 0.001 1.315 0.193 -0.000 0.002
x2 0.2610 0.082 3.167 0.002 0.097 0.425
=====
Omnibus: 118.910 Durbin-Watson: 0.602
Prob(Omnibus): 0.000 Jarque-Bera (JB): 6.966
Skew: 0.016 Prob(JB): 0.0307
Kurtosis: 1.455 Cond. No. 460.
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

sum_sq df F PR(>F)
C(Temp) 1.304119 18.0 0.604321 0.879469
Residual 6.114310 51.0 NaN NaN

WARNING:tensorflow:From C:\Users\jahnavi\AppData\Local\Programs\Python\Python37\lib\site-packages\keras\backend\tensorflow_backend.py:4070: The name tf.nn.max_pool is deprecated. Please use tf.nn.max_pool2d instead.

2025-05-01 09:58:34.731791: I tensorflow/core/platform/cpu_feature_guard.cc:142] Your CPU supports instructions that this TensorFlow binary was not compiled to use: AVX2
WARNING:tensorflow:From C:\Users\jahnavi\AppData\Local\Programs\Python\Python37\lib\site-packages\keras\backend\tensorflow_backend.py:422: The name tf.global_variables is deprecated. Please use tf.compat.v1.global_variables instead.

Activate Windows
Go to Settings to activate Windows.
```

15. CONCLUSION

This project presents an effective method for estimating the remaining shelf life (RSL) of fresh fruits and vegetables during transportation using simulation datasets. By applying the ANOVA algorithm, the study initially achieves moderate prediction accuracy with a 30% error rate. To enhance performance, a deep learning-based extension using CNN is introduced, significantly reducing the prediction error to just 0.26%. The CNN model's ability to extract optimized features improves estimation accuracy and supports better logistics and supply chain decisions. Accurate shelf-life prediction ensures timely delivery and minimizes food waste. The implementation demonstrates that deep learning models can outperform traditional statistical methods in handling real-time shelf-life data, offering a smarter solution for preserving freshness during transit.

16. REFERENCES

- [1] S. K. Srivastava, A. Chaudhuri, and R. K. Srivastava, “Propagation of risks and their impact on performance in fresh food retail,” *Int. J. Logistics Manag.*, vol. 26, no. 3, pp. 568–602, Nov. 2015.
- [2] J. Gustavsson, C. Cederberg, U. Sonesson, R. Van Otterdijk, and A. Meybeck, “Global food losses and food waste,” presented at the Save Food Congr., Düsseldorf, Germany, May 2011. Accessed: Jan. 6, 2022. [Online]. Available: https://www.madr.ro/docs/ind-alimentara/risipa_alimentara/presentation_food_waste.pdf
- [3] Y. Bouzembrak, M. Kluche, A. Gavai, and H. J. P. Marvin, “Internet of Things in food safety: Literature review and a bibliometric analysis,” *Trends Food Sci. Technol.*, vol. 94, pp. 54–64, Dec. 2019. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0924224419303048>
- [4] G. Hough, *Sensory Shelf Life Estimation of Food Products*. Boca Raton, FL, USA: CRC Press, May 2010. [Online]. Available: <https://www.taylorfrancis.com/books/9780429147180>
- [5] S. Jol, A. Kassianenko, K. Wszol, and J. Oggel, “Issues in time and temperature abuse of refrigerated foods,” *Food Saf.*, vol. 11, no. 6, pp. 30–35, 2006.

